

TALLER DE RECURSIVIDAD

Diego Alejandro Flores Serna
Juan Sebastian Martinez Isaza



Universidad del Quindío
Programación III
Armenia - Quindío
04/09/2026

SOLUCIÓN

Punto 1. Número de vocales en una cadena de caracteres:

Forma 1:

```
defmodule ContarVocales do

  def main do
    IO.puts("Ingrese una cadena de texto:")
    texto = IO.gets("") |> String.trim()
    cantidad_vocales = contar_vocales(texto)
    IO.puts("La cantidad de vocales en la cadena es:
#{cantidad_vocales}")
  end

  def contar_vocales(texto) do
    texto
    |> String.downcase()
    |> String.graphemes()
    |> contar_lista()
  end

  defp contar_lista([], do: 0)

  defp contar_lista (head | teil) do
    vocales= ["a", "e", "i", "o", "u"]

    if head in vocales do
      1 + contar_lista(teil)
    else
      contar_lista(teil)
    end
  end
end
```



```

end

defp contar([cabeza | cola]) do
  contar(cola)
end

end

ContarVocalesCharlist.main()

```

warning: using single-quoted strings to represent charlists is deprecated.
 Use ~c"" if you indeed want a charlist or use "" instead.
 You may run "mix format --migrate" to change all single-quoted strings to use the ~c sigil and fix this warning.

```

20   defp contar([cabeza | cola]) when cabeza in 'aeiou' do
                                     ~
    contarVocales2.exs:20:47

```

Cantidad de vocales: 5

Prueba de Escritorio

Fase 1: Subida

head	¿Vocal?	tail	return
“h”	no	["o","l","a"," ","a","m","i","g","o"]	contar_lista(tail)
“o”	sí	["l","a"," ","a","m","i","g","o"]	1 + contar_lista(tail)
“l”	no	["a"," ","a","m","i","g","o"]	contar_lista(tail)
“a”	sí	[" ","a","m","i","g","o"]	1 + contar_lista(tail)
“ ”	no	["a","m","i","g","o"]	contar_lista(tail)
“a”	sí	["m","i","g","o"]	1 + contar_lista(tail)
“m”	no	["i","g","o"]	contar_lista(tail)
“i”	sí	["g","o"]	1 + contar_lista(tail)
“g”	no	["o"]	contar_lista(tail)
“o”	sí	[]	1 + contar_lista([])
[]	-	-	0

Fase 2: Bajada

head	¿Vocal?	Recibe	Calcula	Return
“o”	sí	0	1+0	1
“g”	no	1	1	1
“i”	sí	1	1+1	2
“m”	no	2	2	2
“a”	sí	2	1+2	3
“ ”	no	3	3	3
“a”	sí	3	1+3	4
“l”	no	4	4	4
“o”	sí	4	1+4	5
“h”	no	5	5	5

Punto 2. Verificar Potencia de dos enteros n y b:

```
defmodule VerificarPotencia do

  def main do
    n = "Ingrese un número:"
    |> Util.ingresar(:entero)

    b = "Ingrese otro número:"
    |> Util.ingresar(:entero)

    resultado = verificar_potencia(n, b)
    IO.puts("El resultado es: #{resultado}")

  end

  def verificar_potencia(n, b) do
    cond do
      n == 1 -> true
      n < 1 or b <= 1 -> false
      rem(n, b) != 0 -> false
      true -> verificar_potencia(div(n, b), b)
    end
  end
end
```

```

end
end
end

VerificarPotencia.main()

```

```

1  defmodule VerificarPotencia do
2
3    def main do
4
5      IO.puts("El resultado es: #{resultado}")
6    end
7
8    def verificar_potencia(n, b) do
9      cond do
10        n == 1 -> true
11        n < 1 or b <= 1 -> false
12        rem(n, b) != 0 -> false
13        true -> verificar_potencia(div(n, b), b)
14      end
15    end
16  end
17 end
18
19
20
21
22
23
24 end
25

```

```

PS E:\Trabajos_Universidad\5to_Semestre\Programación III\Talleres\Programacion_III\Preparcial2> elixir VerificarPotencia.e
xs
Ingrese un número:25
Ingrese otro número:5
El resultado es: true
PS E:\Trabajos_Universidad\5to_Semestre\Programación III\Talleres\Programacion_III\Preparcial2> elixir VerificarPotencia.e
xs
Ingrese un número:5
Ingrese otro número:25
El resultado es: false
PS E:\Trabajos_Universidad\5to_Semestre\Programación III\Talleres\Programacion_III\Preparcial2>

```

Prueba de Escritorio

Caso 1: $n = 8$, $b = 2$ (esperado: true, porque $8 = 2^3$):

Llamada	n	b	$n == 1$	$n < 1$ o $b \leq 1$	$\text{rem}(n, b) \neq 0$	Return
verificar_potencia(8, 2)	8	2	false	false	false	(div(8,2)=4, 2)
verificar_potencia(4, 2)	4	2	false	false	false	(div(4,2)=2, 2)
verificar_potencia(2, 2)	2	2	false	false	false	(div(2,2)=1, 2)
verificar_potencia(1, 2)	1	2	true	-	-	true

Punto 3. Verificar Número perfecto

```

defmodule NumeroPerfecto do

  def main do
    numero =

```

```

    "Ingrese un número: "
    |> Util.ingresar(:entero)

    if es_perfecto?(numero) do
      IO.puts("#{numero} es un número perfecto")
    else
      IO.puts("#{numero} NO es un número perfecto")
    end
  end

  def es_perfecto?(numero) when numero > 0 do
    divisores = obtener_divisores(numero, 1, [])
    sumar_divisores(divisores, 0) == numero
  end

  defp obtener_divisores(numero, i, acumulador) when i >= numero do
    acumulador
  end

  defp obtener_divisores(numero, i, acumulador) do
    if rem(numero, i) == 0 do
      obtener_divisores(numero, i + 1, [i | acumulador])
    else
      obtener_divisores(numero, i + 1, acumulador)
    end
  end

  defp sumar_divisores([], acumulador), do: acumulador

  defp sumar_divisores([head | tail], acumulador) do
    sumar_divisores(tail, acumulador + head)
  end
end

NumeroPerfecto.main()

```

```

PS D:\Universidad\Programacion 3\Elixir\Programacion3\Talleres\TallerRecursividad> elixir numeroPerfecto.exs
Texto ingresado correctamente.
Entrada: 81

81 NO es un número perfecto
PS D:\Universidad\Programacion 3\Elixir\Programacion3\Talleres\TallerRecursividad> elixir numeroPerfecto.exs
Texto ingresado correctamente.
Entrada: 6

6 es un número perfecto
PS D:\Universidad\Programacion 3\Elixir\Programacion3\Talleres\TallerRecursividad>

```

Prueba de Escritorio

Ejemplo con el número 6

Fase 1: Subida

i	Acumulador entrada	i >= 6?	rem(6,i)==0?	Acción	Acumulador salida
1	[]	no	sí (rem=0)	[1 []]	[1]
2	[1]	no	sí (rem=0)	[2 [1]]	[2, 1]
3	[2, 1]	no	sí (rem=0)	[3 [2,1]]	[3, 2, 1]
4	[3, 2, 1]	no	no (rem=2)	avanza	[3, 2, 1]
5	[3, 2, 1]	no	no (rem=2)	avanza	[3, 2, 1]
6	[3, 2, 1]	sí	-	retorna lista	[3, 2, 1]

Fase 2: Bajada

head	tail	Acumulador entrada	Acción	Acumulador salida
3	[2, 1]	0	llama ([2,1], 3)	0+3 = 3
2	[1]	3	llama ([1], 5)	3+2 = 5
1	[]	5	llama ([], 6)	5+1 = 6
[]- Caso Base	-	6	retorna acumulador	6

Punto 4. Cadena de texto más larga

```
defmodule ListaCadenas do

  def main do
    lista = ["Hola", "Mundo", "Bienvenidos a Elixir", "Es", "Genial",
"Gorila"]

    resultado = mas_larga(lista)
    IO.puts("La cadena más larga es: #{resultado}")
  end

  def mas_larga([h]), do: h

  def mas_larga([h | t]) do
    comparar(h, mas_larga(t))
  end
end
```

```

end

defp comparar(a, b) do
  if String.length(a) >= String.length(b) do
    a
  else
    b
  end
end

end

ListaCadenas.main()

```

```

PS E:\Trabajos_Universidad\5to_Semestre\Programación III\Talleres\Programacion_III> cd Preparcial2
PS E:\Trabajos_Universidad\5to_Semestre\Programación III\Talleres\Programacion_III\Preparcial2> elixir TextoLargo.exs
La cadena más larga es: Bienvenidos a Elixir
PS E:\Trabajos_Universidad\5to_Semestre\Programación III\Talleres\Programacion_III\Preparcial2>

```

Prueba de Escritorio

Fase 1: Subida

h	length	t	Return
"Hola"	4	["Mundo","Bienvenidos a Elixir","Es","Genial","Gorila"]	mas_larga(t)
"Mundo"	5	["Bienvenidos a Elixir","Es","Genial","Gorila"]	mas_larga(t)
"Bienvenidos a Elixir"	20	["Es","Genial","Gorila"]	mas_larga(t)
"Es"	2	["Genial","Gorila"]	mas_larga(t)
"Genial"	6	["Gorila"]	mas_larga(t)
"Gorila"	6	-	"Gorila"

Fase 2: Bajada

Comparar(a,b)	length(a)	length(b)	$a \geq b$?	Return
comparar("Genial", "Gorila")	6	6	sí ($6 \geq 6$)	"Genial"
comparar("Es", "Genial")	2	6	no ($2 < 6$)	"Genial"

comparar("Bienvenidos a Elixir", "Genial")	20	6	sí ($20 \geq 6$)	"Bienvenidos a Elixir"
comparar("Mundo", "Bienvenidos a Elixir")	5	20	no ($5 < 20$)	"Bienvenidos a Elixir"
comparar("Hola", "Bienvenidos a Elixir")	4	20	no ($4 < 20$)	"Bienvenidos a Elixir"

Punto 5. Número Reversible

```
defmodule NumeroReversible do

  def main do
    n = 36

    case es_reversible?(n) do
      true  -> IO.puts("#{n} es reversible")
      false -> IO.puts("#{n} NO es reversible")
    end
  end

  def es_reversible?(n) when is_integer(n) and n > 0 do
    invertido = invertir(n, 0)
    suma = n + invertido

    todos_impares?(suma)
  end

  def es_reversible?(_, do: false)

  defp invertir(0, acc), do: acc

  defp invertir(n, acc) do
    digito = rem(n, 10)
    nuevo_acc = acc * 10 + digito
    invertir(div(n, 10), nuevo_acc)
  end

  defp todos_impares?(0), do: true

  defp todos_impares?(n) do
    digito = rem(n, 10)
  end
end
```

```

        if rem(digito, 2) == 0 do
            false
        else
            todos_impares?(div(n, 10))
        end
    end
end

NumeroReversible.main()

```

```

1  defmodule NumeroReversible do
2
3      def main do
4          n = 57
5
6          case es_reversible?(n) do
7              true  -> IO.puts("#{n} es reversible")
8              false -> IO.puts("#{n} NO es reversible")
9          end
10
11      end
12 end

```

PROBLEMS 5 OUTPUT SPELL CHECKER 5 DEBUG CONSOLE TERMINAL PORTS

```

PS D:\Universidad\Programacion 3\Elixir\Programacion3\Talleres\TallerRecursividad> elixir punto5.exs
36 es reversible
PS D:\Universidad\Programacion 3\Elixir\Programacion3\Talleres\TallerRecursividad> elixir punto5.exs
57 NO es reversible
PS D:\Universidad\Programacion 3\Elixir\Programacion3\Talleres\TallerRecursividad>

```

Fase 1

Pas o	n (36)	acc (36)	digi to	acc nuevo	n sig	acc final	n (57)	acc (57)	digi to	acc nuevo	n sig	acc final
1	36	0	6	6	3	6	57	0	7	7	5	7
2	3	6	3	63	0	63	5	7	5	75	0	75
3	0	63	-	-	-	63	0	75	-	-	-	75

Fase 2

Número	Operación	Resultado
36	36 + 63	99
57	57 + 75	132

Fase 3.

Pas o	n (36)	digit o	¿par ?	Resultado parcial	n sig	n (57)	digit o	¿par ?	Resultado parcial	n sig
1	99	9	No	true	9	132	2	Sí	false	-
2	9	9	No	true	0	-	-	-	-	-
3	0	-	-	true	-	-	-	-	-	-

Resultado Final:

Número	¿Reversible?	Razón
36	TRUE	99 → todos impares
57	FALSE	132 → tiene 2 (par)

REFLEXIÓN DE LA IA

El uso de la inteligencia artificial en el desarrollo de este taller fue una buena herramienta de apoyo para comprender mejor los conceptos de recursividad en programación. Con su ayuda, fue posible aclarar dudas, profundizar algunos conceptos con ejemplos y obtener explicaciones más detalladas acerca de las funciones propias de Elixir como String.graphemes y la syntaxis. Debemos utilizar este tipo de herramientas de manera responsable y solo como apoyo, ya que la lógica no se le puede delegar a una máquina o terminamos siendo dependientes de esta misma.

CONCLUSIONES GENERALES

En conclusión, el desarrollo de esta actividad permitió fortalecer la comprensión de la recursividad como una herramienta fundamental en la programación. A través de los ejercicios propuestos, logramos mejorar la lógica de programación y el análisis del comportamiento de los algoritmos, también en el manejo de listas y cadenas en Elixir. Además, el realizar las pruebas de escritorio nos permitió comprender mejor el proceso de ejecución de la pila.